**WPI**

WORCESTER POLYTECHNIC INSTITUTE
ROBOTICS ENGINEERING PROGRAM

Lab 4 – Velocity Kinematics

SUBMITTED BY
Sukriti Kushwaha
Istan Slamet
Tracy Yang

Date Submitted: September 28, 2023

Date Completed: September 28, 2023

Course Instructor: Dr. Agheli

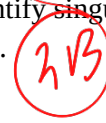
Lab Section: RBE 3001 A'12

Abstract

This report overviews the experiments performed in Lab 4, which were used to learn how to utilize differential kinematics and to move an OpenManipulator-X robotic arm. The experiments primarily included analysis of Jacobian matrices to calculate velocity vectors and avoid singularities. All experiments were conducted using MATLAB.

Introduction

Using an accumulation of what we learned throughout the term, we implemented velocity kinematics in the robot arm. We calculated the Jacobian matrix to solve for the linear and angular velocities in the task space. From there, we completed a triangular trajectory in the task space using speed and direction commands and plotted the stick model of the robot with its velocity vector pointing in the direction of the end effector. We also utilized the determinant of the Jacobian matrix to identify singular configurations of the robot and apply an emergency stop as it approached such configurations.



Methodology

To solve the Jacobian matrix, we needed to define the homogenous transformation matrices of each joint in symbolic form. The symbolic form allowed us to differentiate the matrices with respect to the initial time or final time. By doing so, we calculated the linear velocity with MATLAB's `jacobian()` function. For the angular velocity, we multiplied the linear velocities of each joint with the previous joint's velocity. We have now solved each joint's coefficients of the initial angular and linear velocities as well as the final of the two using the current joint angles. In the end, this method returned a 6 by 4 matrix, where the top 3 rows were the linear velocity of the 3 axes, the bottom 3 were the angular velocities of the 3 axes, and the columns corresponded to joints 1 through 4.

Using this method, we were able to calculate the current and updated forward velocity kinematics when the robot arm moves. We wrote the function `fdk3001()`, which took in the joint angles and returned a 6 by 1 vector with all the linear and angular velocities in the task space. With these vectors, we plotted a stick model that updated in real-time displays the linear velocity vector, the length of which changed proportionally to the magnitude of the end effector's velocity. The magnitude changed using the method we created in the first lab, `measure_js()`, and we retrieved the current joint angles and velocities and input them into our forward velocity kinematics function `fdk3001()`. We tested this by executing a trajectory path similar to the previous lab's. We confirmed that the robot arm could live plot and show its end effector's velocity vector correctly.

The last implementation we needed to add was to set boundaries for the robot arm. We set up an emergency stop to prevent it from going into a singular configuration (where the determinant of the Jacobian is 0). The method `jacob3001()` considered the determinant when it changed configurations, and when it reached a range near 0 (when not considering offsets and its translation to radians), it threw an error and stopped plotting. For our determinant bounds, we threw an error if the calculated determinant was under a magnitude of 20,000. This prevented the robot arm from going out of bounds or near the camera.

Results

Needed to write out jacobian
formula w/ partial derivatives
(symbolic, such as $\frac{\partial \theta}{\partial k}$)

-1

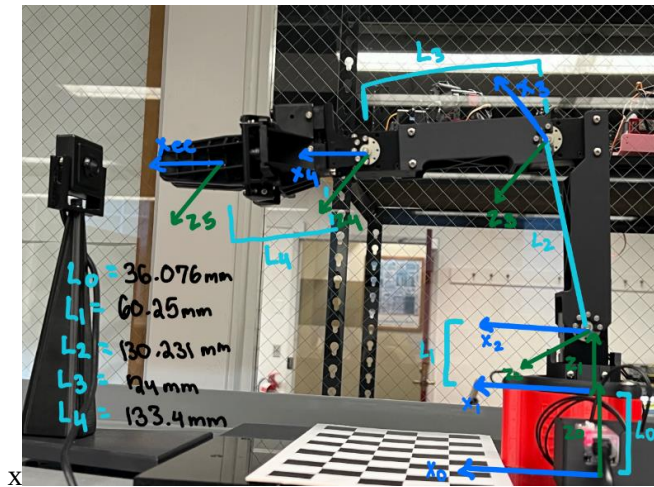


Figure 1: A picture of the robot arm, its joint axes, and its link lengths

	θ	d	a	α
F_0^1	0	L_0	0	0
F_1^2	0^*	L_1	0	$-\pi/2$
F_2^3	$-\tan^{-1}(\frac{128}{24}) + 0^*$	0	L_2	0
F_3^4	$\tan^{-1}(\frac{128}{24}) + 0^*$	0	L_3	0
F_4^{ee}	0^*	0	L_4	0

Figure 2: A table of the robot arm's DH Parameters

```

Command Window

mat =

    0.0000    385.6771    256.7192    133.0400
    0.5487     0.0000     0.0000     0.0000
     0    -0.5487    -18.7070    -9.7934
     0         0         0         0
     0     1.0000     1.0000     1.0000
    1.0000     0.0000     0.0000     0.0000

Error using Robot/run_trajectory (line 438)
Singular position!

f3 Error in Lab4 (line 220)
  
```

Figure 3: A screenshot of the results of jacob3001() in the command window

What are the
joint values
for these examples?

-1

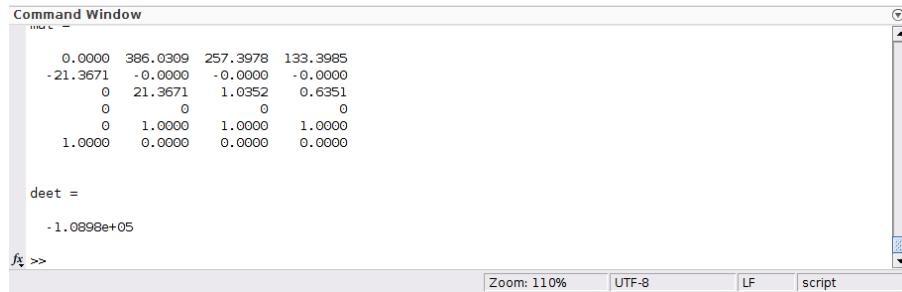
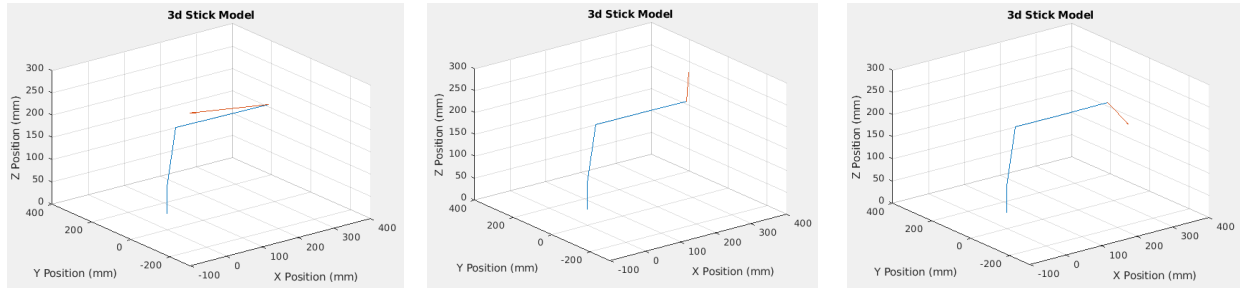


Figure 4: A screenshot of the results of jacob3001() determinant in the command window



Figures 5, 6, and 7: The stick model plots of each position (vertex) that is in the triangular trajectory

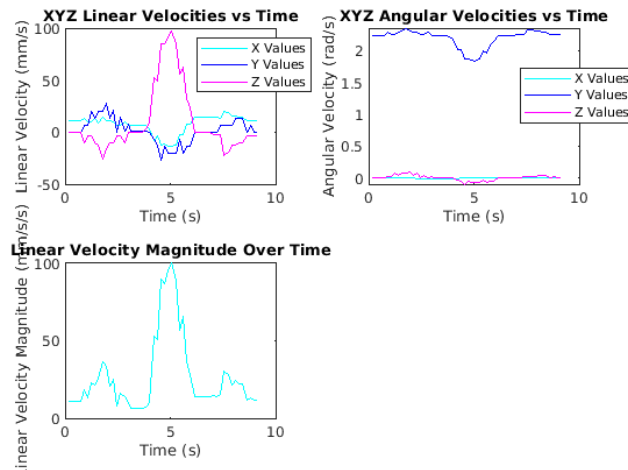


Figure 8: A plot with subplots depicting the linear velocities in the X, Y, and Z, the angular velocities in the X, Y, and Z, and the magnitude of the linear velocity all in the complete triangular trajectory (from top left to the bottom)

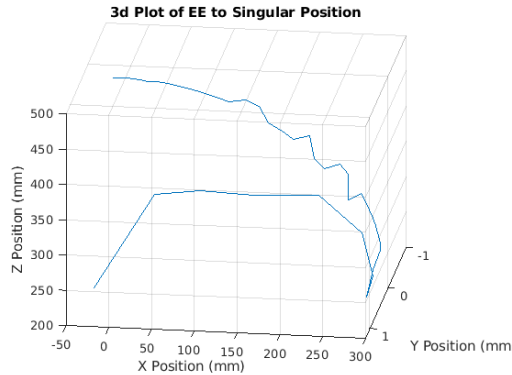


Figure 9: A 3D plot of the path the robot arm takes when going from home to the singular configuration

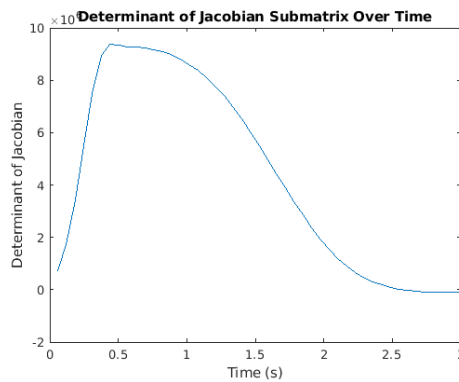


Figure 10: A plot of the determinant values versus time when going from home to the singular configuration

Discussion

When sending the robot's joint angles to the values $[0, -10, -80, 0]$, the determinant of the Jacobian matrix should ideally approach zero due to it being in a singular configuration. The determinant of the matrix becomes zero because there are infinite inverse kinematic solutions, meaning the same position can be achieved with different joint orientations. Additionally, the values of the first column of the determinant for our robot are also close to zero because that joint loses its motion. This can be seen in Figure 10, where it approaches 0. Since our calculations for the Jacobian are in radians, however, the determinant for the matrix is extremely large, in the range of 10^6 . This means that when our robot is in a singular configuration, the determinant is actually close to 10^3 . Singularities occur when the joint axes are aligned, or the robot is approaching the edge of the reachable task space. Having such an emergency stop is important because a robot can lose its degrees of freedom and its movements can become fast enough to cause damage to the robot. We implemented a threshold so that if the determinant was either less than 20,000 or greater than $-20,000$, then the robot stopped its motion and displayed an error.

When plotting the stick model of the robot with the velocity vector as it traverses the triangular trajectory, we noticed that the direction of the velocity changed as the end effector changed its position in the workspace. The velocity vector always points towards the direction of motion of the end effector as seen in Figures 5 through 7.

9/12

Conclusion:

After completing the last lab before the project, we successfully solved and plot the forward velocity kinematics. Our knowledge of the Jacobian matrix and its application in this lab has allowed us to perform trajectory following dependent on speed and direction. The accumulation of MATLAB functions and course materials prepared us fully for this lab. We also were able to live-plot the arm as well as the linear velocity vector of the end effector. This lab also allowed us to understand the dangers of singular configurations and implement ways to stop them through analysis of the determinant.

413

Appendix A: Authorship

Section	Author
Introduction	Tracy Yang and Sukriti Kushwaha
Methodology	Tracy Yang and Istan Slamet
Results	Tracy Yang
Discussion	Sukriti Kushwaha and Istan Slamet
Conclusion	Everyone